

What the platform does today

The product as it is actually built, feature by feature, with what is finished, what is dark, and what was deliberately left out. Written from the code, not from the plan.

COMMIT

7bee05e

SURFACE

206 endpoints, 3 clients

WRITTEN

1 October 2026

VERIFIED AGAINST

the repository

Contents

- What this is** 2
 - In one paragraph 2
 - The three clients 3
 - What the product has to survive 3
- The student's product** 3
 - Getting in 4
 - What a student can see 4
 - Sitting a paper 4
 - Getting marked 5
 - Reading the result 5
 - The rest of the student's account 5
- The institute's product** 5
 - Who the staff are 7
 - Building a paper 7
 - The question bank 8
 - Running a test day 8
 - Roster and catalog 8
- What is not built** 9
- One thing that was blocking, and is not now** 10

What this is

In one paragraph

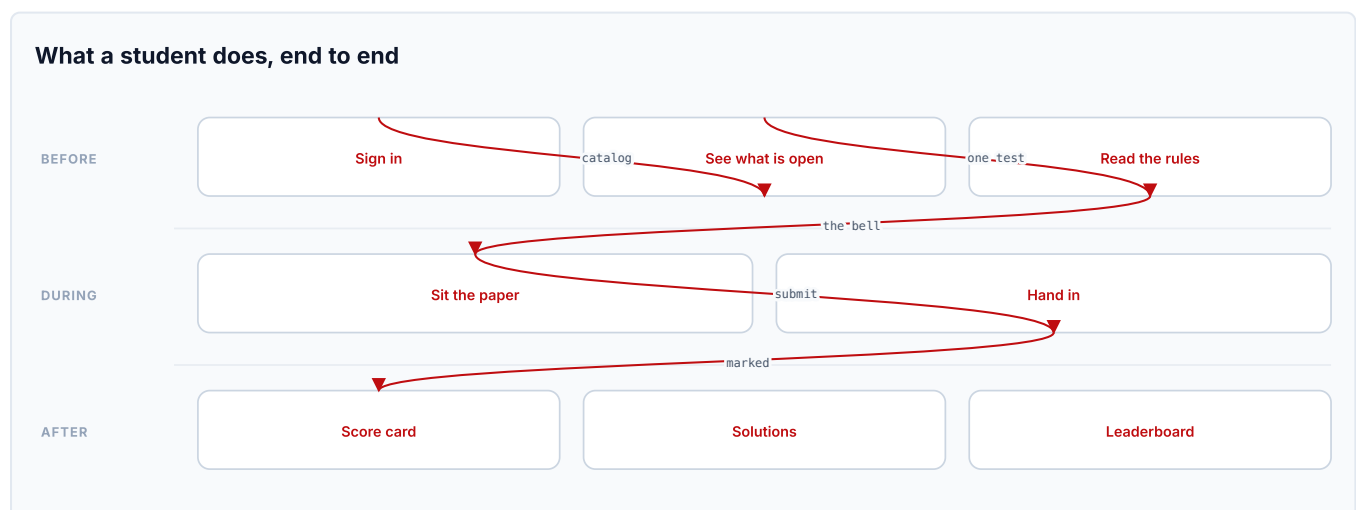
A mock-test platform for a government-exam coaching institute. Staff build papers from a reusable blueprint, either by writing the questions or by drawing them from a bank; two people type and proof-read every paper before it is offered. Students sit those papers under exam conditions, on a clock, and are marked within seconds of handing in. They then get a score card, their rank and percentile against everyone who sat the same paper, a worked solution for every question, and a record of how they are trending. It runs on the web for staff, on the web for students, and on an Android app that is built but not launched.

The three clients

CLIENT	WHO USES IT	STATE
Admin portal — apps/admin	Institute staff: owners, typists, proof-readers	Built. 141 of the 206 endpoints are its
Student portal — apps/exams	Candidates, on a laptop or phone browser	Built, and the only way to sit a paper today
Mobile app — apps/mobile	Candidates, Android first	Screens complete; not launched — no API is deployed, push is dark, icons are placeholders

What the product has to survive

One number shapes almost every technical decision in this platform: **three thousand students sitting the same paper in the same minute, with headroom to eight thousand**. A mock test is a synchronised event — everybody opens at the bell and most hand in within a few minutes of each other. That is why the timer runs in the browser, why answers are held in memory and written to the database on a timer rather than per keystroke, and why marking happens on a queue after the fact rather than in the request that submits the paper.



The student's product

Getting in

A student signs in with their **mobile number and a four-digit PIN**. The institute issues the PIN when the student is added to the roster and texts it to them; the first thing the platform asks is that they replace it, and until they do, every screen behind the shell is replaced by a change-PIN card. A student who has forgotten it requests a one-time code by SMS, and setting a new PIN is what mints their session.

Two deliberate product decisions sit here. The PIN is **not unique across students** — it is only ever checked against the one student a mobile number resolves to, because requiring uniqueness would cap the platform at ten thousand students. And a student gets **one session per kind of device**: signing in on a second phone ends the first phone's session, while a browser session continues alongside it.

What a student can see

A student does not browse a catalogue of everything the institute sells. They see exactly the test series that reach them, by one of four routes, plus anything granted to them by hand:

HOW A SERIES REACHES A STUDENT	WHAT IT DEPENDS ON
Free	Reaches every active student
Standard	Their branch is on the series' list and their enrolled course matches the series' exam
Program	They hold the program's code
Event	They are on that event's candidate roster
Grant	An admin gave this one student this one series

A series can be **sequential**, meaning its tests must be sat in order — the next one does not open until the previous is done. Individual tests have an opening time, and a program can be given an earlier one, so the same paper can open for one cohort before another.

Note This is recomputed on every read rather than stored, so a change to a student's branch, enrolment or grants takes effect on their next screen refresh with nothing to invalidate.

Sitting a paper

The sitting is the product. It replicates a government computer-based test closely enough that the hall is not a surprise: a question panel, an option list, a palette of every seat colour-coded by state, a section bar, and a countdown.

What it does:

- **The paper is fetched before the student presses Begin**, so the bell does not become a stampede on the server.
- **The clock is granted by the server and counted down by the browser**. The student's device clock is never trusted; the server says how long is left and when it ends.
- **Answers are kept locally and sent in batches** roughly every 25 seconds, or sooner if the student is answering quickly. A failed send never takes an answer off the screen, and an offline tab keeps its queue and resends on reconnect.
- **One sitting at a time**. Opening the same paper in a second tab stands the first one down rather than letting two tabs fight; the stood-down tab offers "Continue here".
- **Leaving is guarded**. Closing the tab with unsent answers warns first, and sends one last batch on the way out.
- **A reload is survivable**, and a student who walks away gets their clock credited for the time they were gone, up to a ceiling.

Papers can be **forward-only**, where a student cannot go back to a seat they have passed. That is enforced on both the screen and the server, so a crafted request cannot change an answer behind the furthest point reached.

Note One honest limit: if a student answers on two devices at once, the second device's queue can overwrite the first's. A per-question version on the server is the fix; a client-side attempt at it was built and reverted because the client cannot judge it correctly.

Getting marked

Handing in is one request. The server records the submission and queues the marking; the student goes to a screen that says marking is in progress. Marking itself happens on a worker: it reads the answer sheet, compares each answer to the key, applies the paper's marks and negative marks, and writes the result. A question the staff later **drop** gives its marks back to anyone who attempted it; a **bonus** question gives marks to everyone. Changing a question's disposition re-marks every sitting of that paper automatically.

If marking fails or is lost, a sweep notices within two minutes and asks again. A re-sent submit cannot produce a second result — the submission is claimed once and the marking job is keyed to the sitting.

Reading the result

SCREEN	WHAT IT ANSWERS
Score card	What did I score, out of what, and where does that put me
Sections	Which parts of the paper carried me and which cost me
Solutions	Why is each answer right — stem, options and a worked solution, one section at a time
Questions	Every question with my answer, the correct one, my time, and how the cohort did
Compare	Me against the cohort and against the topper
Leaderboard	The top three, and the seats immediately around mine
Performance	My trend across sittings, and my accuracy by subject

Rank and percentile are counted live. They are not stored and there is no regeneration step — the moment the next student is marked, every rank is correct, because the rank is a count rather than a record.

Watch A student whose paper is marked can open the answer key while others are still writing, and can pass it on. There is no cohort-wide release instant in the product — the gate is "your own sitting, and it has been marked". This was a deliberate trade when the solutions gate was removed, and it is worth re-deciding before a public rollout rather than after.

The rest of the student's account

Saved questions (starring anything from a marked sitting for revision), an in-app notification bell, a profile with a photo and a 10th marksheet, a PIN change, and a list of signed-in devices with the ability to sign one out. **Identity documents are never stored** — Aadhaar and PAN exist in the product only as verification flags, and even those have no screen that can set them today.

The institute's product

Who the staff are

Staff are not students with extra rights; they are a separate kind of account entirely, and they **sign in with an emailed one-time code every time** — there is no staff password anywhere in the platform. What a staff member may do is a set of named permissions, each at read, write or export level. A role (typist, proof-reader, admin, super admin) is a convenience that pre-fills those permissions when the account is made.

Watch Changing a staff member's role later does **not** change the permissions they already hold. Promote someone to super admin and then set them back to typist, and the original admin permissions are still there and live again. A super admin's role also cannot be changed from the UI at all — the screen that would do it is hidden for exactly the people who need it.

Building a paper

This is the deepest part of the product and the part most unlike an off-the-shelf tool.

A blueprint, not a template. A base config describes the shape of an exam paper: its sections, how many questions each holds, the marks and negative marks, the duration, how the timer behaves (whole paper, per section, or per question), which languages, and which on-screen skin. Tests are cut from a blueprint, and many tests share one.

A test is one instance of a blueprint, with its own title, its own series, and optionally a narrower scope — one module or one section of the blueprint rather than all of it.

Two ways to fill a paper:

	PICKED	FRAMED
Where questions come from	Drawn from the bank	Written for this paper
Who fills it	The test owner, by drawing or hand-picking	An assigned typist
The draw	Seeded and repeatable, honouring a topic and difficulty mix	—

Every paper is read before it is offered. A section is assigned to a typist and a proof-reader. The typist writes or chooses; pressing Done hands the section to the reader and makes the selection the paper. The reader ticks each question, or sends one back with a reason; a sent-back question becomes editable again even after Done, and the typist marks it fixed. A test cannot be offered until every seat's work is finished and every question on a read section carries its tick — unless a super admin overrides it.

Offering freezes the paper. From that moment the database itself refuses changes: triggers, not application code, stop a paper or a question's content moving once anybody has sat it. The one thing that can still change is a question's disposition — dropping or bonusing it — which re-marks every sitting.

Watch A frozen paper cannot be un-frozen. A test can be made inactive, which hides it and stops new sittings, but the paper stays frozen permanently. There is no thaw.

The question bank

Questions live under a subject and topic taxonomy with free-text tags, in up to three languages (English, Hindi, Telugu), with rich content — figures and mathematical formulas included. Each question has versions: a paper pins the exact version it served, so correcting a typo can never change what a student already sat.

The rule for when an edit rewrites the existing version and when it creates a new one turns on whether any paper that pins it is **reachable by a student** — finalized and open, or within five minutes of opening. If none is, the edit is made in place; if one is, a new version is written. That is the single most consequential rule in the bank, because answer sheets store option positions rather than option ids.

Questions can be imported in bulk from a spreadsheet, with a preview that reports errors per row, a correction window, and a commit that writes only valid rows.

Running a test day

CAPABILITY	WHAT IT DOES
Live board	Watch a test in progress — who has started, who is still writing
Force submit	End one student's sitting
Extend	Give one student more time
Reset / void	Let a student sit again, or discard a sitting from the rankings
Analytics	Per-test and per-question performance across the cohort
Announcements	A message to a filtered set of students, in-app and optionally by SMS or WhatsApp with a cost estimate shown before sending
Audit	Who changed what, with before-and-after values, and a downloadable record of every bulk import

Roster and catalog

Students are added one at a time or imported by spreadsheet, with their branch, their enrolled exams and courses, and their programs. Branches are physical or virtual — an online student sits in the single virtual branch. A student can be deactivated (cannot sign in), blocked from tests (can sign in, cannot start one), or **erased**, which anonymises their personal data in place while keeping their sittings, because a cohort's statistics must not move when one person exercises their rights.

Erasure now removes the uploaded photo and the 10th marksheet from storage as well as the database references, and it removes them **before** the row, so an erasure that reports success cannot have left the files behind. Storage deletes are idempotent, so a retry after a part-way failure is clean.

What is not built

Honest scope. Each of these is a deliberate absence, not an oversight.

NOT BUILT	STATUS
Mobile push notifications	The whole path is written and tested; it is dark because Firebase credentials are not configured and the key file is not in the repository. The in-app bell works everywhere.
The mobile app's launch	Screens are complete, the exam sitting included. No API is deployed, so only the development build works. The exam sitting has never been verified on a physical device.
A broad student portal	V1 is the mock-test feature. Course content, doubt-solving and the rest are later.
Identity verification	Aadhaar and PAN flags exist; nothing can set them. Images are deliberately never stored.
A solutions release gate	Removed on purpose; see the note above.
Re-scoring one sitting on demand	The only lever is changing a question's disposition, which re-marks the whole cohort.

One thing that was blocking, and is not now

The deployment was blocked: the API's container image still referenced a directory renamed on 30 September, so it could not build and no release could be cut. That is **fixed** — the image now builds end to end, verified against all three of its targets. The engineering document's findings ledger carries the detail, and everything else in that ledger is still open.